

Multi-Vendor E-Commerce Marketplace - Complete System Architecture

Project: Jumia-like Multi-Vendor Marketplace

Stack: Node.js (NestJS + TypeScript), MongoDB + PostgreSQL, Redis

Architecture: Modular Monolith (Domain-Driven Design)

Table of Contents

1. System Overview
 2. High-Level Architecture
 3. Database Design
 4. REST API Design
 5. Module Structure
 6. Third-Party Integrations
 7. Security & Authentication
 8. Deployment Strategy
-

1. System Overview

Business Rules Summary

- **Commission Model:** Vendor subscription fee + smaller commission per sale
- **Vendor Onboarding:** Manual admin approval with business document verification
- **Payout Schedule:** Monthly batch payouts (1st of every month)
- **Product Model:** Multi-SKU with variants (Color, Size, Material, etc.)
- **Order Flow:** Single order with sub-orders per vendor
- **Payment Methods:** Card (Paystack/Flutterwave), Bank Transfer, Cash on Delivery
- **Refund Window:** 14 days post-delivery
- **Delivery:** Hybrid model (vendor's own logistics or platform-provided)
- **Ratings:** Product, Vendor, and Delivery experience (all separate)

User Roles

1. **Customer** - Browse, purchase, review, manage orders
 2. **Vendor** - Manage products, inventory, orders, view analytics
 3. **Admin** - Platform oversight, vendor approval, dispute resolution, analytics
-

2. High-Level Architecture

CLIENT APPLICATIONS
Web App (React/Next.js) Mobile App Admin Dashboard

HTTPS / REST API

API GATEWAY / LOAD BALANCER
(Nginx / AWS ALB)

NestJS MODULAR MONOLITH

Auth Module	Products Module	Orders Module	Payments Module
----------------	--------------------	------------------	--------------------

Vendors Module	Users Module	Reviews Module	Search Module
-------------------	-----------------	-------------------	------------------

Admin Module	Analytics Module	Notificat ions	Media Module
-----------------	---------------------	-------------------	-----------------

PostgreSQL	MongoDB	Redis
- Users	- Products	- Sessions
- Orders	- Reviews	- Carts
- Payments	- Search	- Cache
- Vendors	Index	- Queues
- Payouts		

THIRD-PARTY SERVICES

Paystack/Flutterwave (Payments)	AWS S3/Cloudinary (Media Storage)	SendGrid (Email)
Termii (SMS)	DHL/GIG Logistics (Delivery API)	Elasticsearch (Search Engine)

Why Modular Monolith?

1. **Single deployable unit** - Easier DevOps, lower infrastructure cost
 2. **Shared database transactions** - ACID guarantees across modules
 3. **Clear module boundaries** - Can extract to microservices later if needed
 4. **Faster development** - No inter-service communication overhead
 5. **Easier debugging** - Single codebase, single log stream
-

3. Database Design

Database Split Strategy

PostgreSQL (Transactional Data) - Users, Vendors, Orders, Payments, Payouts - Requires ACID compliance, complex joins, referential integrity

MongoDB (Flexible Schema Data) - Products (with dynamic attributes per category) - Product Reviews & Ratings - Search indexes, Logs

Redis (In-Memory Cache) - User sessions, JWT blacklist - Shopping carts (temporary data) - Rate limiting counters - Real-time inventory locks

PostgreSQL Schemas

1. Users Table

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  phone VARCHAR(20),  
  password_hash VARCHAR(255), -- NULL for guest checkouts converted to accounts  
  first_name VARCHAR(100),  
  last_name VARCHAR(100),  
  role VARCHAR(20) NOT NULL CHECK (role IN ('customer', 'vendor', 'admin')),  
  is_email_verified BOOLEAN DEFAULT FALSE,  
  is_phone_verified BOOLEAN DEFAULT FALSE,  
  status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'suspended', 'deleted'))
```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

```

```

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);

```

2. Vendors Table

```

CREATE TABLE vendors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    business_name VARCHAR(255) NOT NULL,
    business_email VARCHAR(255),
    business_phone VARCHAR(20),
    business_address TEXT,
    business_registration_number VARCHAR(100),
    tax_id VARCHAR(100),
    bank_account_name VARCHAR(255),
    bank_account_number VARCHAR(50),
    bank_name VARCHAR(100),
    verification_status VARCHAR(20) DEFAULT 'pending' CHECK (
        verification_status IN ('pending', 'approved', 'rejected', 'suspended')
    ),
    subscription_plan VARCHAR(50) DEFAULT 'basic' CHECK (
        subscription_plan IN ('basic', 'premium', 'enterprise')
    ),
    subscription_start_date DATE,
    subscription_end_date DATE,
    commission_rate DECIMAL(5, 2) DEFAULT 5.00, -- 5% commission
    total_sales DECIMAL(15, 2) DEFAULT 0.00,
    total_products INTEGER DEFAULT 0,
    rating DECIMAL(3, 2) DEFAULT 0.00, -- Aggregate vendor rating
    total_reviews INTEGER DEFAULT 0,
    rejected_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE INDEX idx_vendors_user_id ON vendors(user_id);
CREATE INDEX idx_vendors_status ON vendors(verification_status);

```

3. Vendor Documents Table

```

CREATE TABLE vendor_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

    vendor_id UUID REFERENCES vendors(id) ON DELETE CASCADE,
    document_type VARCHAR(50) NOT NULL CHECK (
        document_type IN ('business_license', 'tax_certificate', 'id_proof', 'bank_statement')
    ),
    document_url TEXT NOT NULL,
    verification_status VARCHAR(20) DEFAULT 'pending' CHECK (
        verification_status IN ('pending', 'verified', 'rejected')
    ),
    verified_by UUID REFERENCES users(id), -- Admin who verified
    verified_at TIMESTAMP,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

4. Categories Table

```

CREATE TABLE categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    slug VARCHAR(100) UNIQUE NOT NULL,
    parent_id UUID REFERENCES categories(id) ON DELETE CASCADE,
    created_by UUID REFERENCES users(id), -- Admin or Vendor
    is_admin_created BOOLEAN DEFAULT TRUE,
    image_url TEXT,
    description TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE INDEX idx_categories_parent ON categories(parent_id);
CREATE INDEX idx_categories_slug ON categories(slug);

```

5. Orders Table

```

CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_number VARCHAR(50) UNIQUE NOT NULL, -- e.g., ORD-20260216-0001
    customer_id UUID REFERENCES users(id),
    customer_email VARCHAR(255) NOT NULL, -- For guest checkouts
    customer_phone VARCHAR(20),
    total_amount DECIMAL(15, 2) NOT NULL,
    platform_commission DECIMAL(15, 2) DEFAULT 0.00,
    payment_method VARCHAR(50) NOT NULL CHECK (
        payment_method IN ('card', 'bank_transfer', 'cash_on_delivery', 'wallet')
    ),
    payment_status VARCHAR(20) DEFAULT 'pending' CHECK (

```

```

        payment_status IN ('pending', 'paid', 'failed', 'refunded', 'partially_refunded')
    ),
    order_status VARCHAR(20) DEFAULT 'pending' CHECK (
        order_status IN ('pending', 'confirmed', 'processing', 'shipped', 'delivered', 'cancelled')
    ),
    shipping_address TEXT NOT NULL,
    billing_address TEXT,
    payment_reference VARCHAR(255), -- Paystack/Flutterwave reference
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_number ON orders(order_number);
CREATE INDEX idx_orders_status ON orders(order_status);
CREATE INDEX idx_orders_payment_status ON orders(payment_status);

```

6. Order Items (Sub-Orders per Vendor)

```

CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id) ON DELETE CASCADE,
    vendor_id UUID REFERENCES vendors(id),
    product_id UUID NOT NULL, -- MongoDB reference
    sku_id UUID NOT NULL, -- MongoDB reference
    product_name VARCHAR(255) NOT NULL, -- Denormalized for order history
    sku_attributes JSONB, -- e.g., {"color": "Red", "size": "M"}
    quantity INTEGER NOT NULL,
    unit_price DECIMAL(15, 2) NOT NULL,
    subtotal DECIMAL(15, 2) NOT NULL,
    vendor_commission DECIMAL(15, 2),
    delivery_status VARCHAR(20) DEFAULT 'pending' CHECK (
        delivery_status IN ('pending', 'processing', 'shipped', 'delivered', 'returned', 'cancelled')
    ),
    tracking_number VARCHAR(100),
    courier_service VARCHAR(100),
    shipped_at TIMESTAMP,
    delivered_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_order_items_vendor ON order_items(vendor_id);
CREATE INDEX idx_order_items_delivery_status ON order_items(delivery_status);

```

7. Payments Table

```
CREATE TABLE payments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_id UUID REFERENCES orders(id) ON DELETE CASCADE,  
  amount DECIMAL(15, 2) NOT NULL,  
  payment_method VARCHAR(50) NOT NULL,  
  payment_gateway VARCHAR(50), -- 'paystack', 'flutterwave'  
  transaction_reference VARCHAR(255) UNIQUE,  
  status VARCHAR(20) DEFAULT 'pending' CHECK (  
    status IN ('pending', 'successful', 'failed', 'cancelled', 'refunded')  
  ),  
  gateway_response JSONB, -- Store full gateway response  
  metadata JSONB,  
  paid_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_payments_order ON payments(order_id);  
CREATE INDEX idx_payments_reference ON payments(transaction_reference);
```

8. Refunds Table

```
CREATE TABLE refunds (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_id UUID REFERENCES orders(id),  
  order_item_id UUID REFERENCES order_items(id),  
  customer_id UUID REFERENCES users(id),  
  vendor_id UUID REFERENCES vendors(id),  
  refund_amount DECIMAL(15, 2) NOT NULL,  
  refund_reason TEXT NOT NULL,  
  refund_type VARCHAR(20) CHECK (refund_type IN ('full', 'partial')),  
  status VARCHAR(20) DEFAULT 'pending' CHECK (  
    status IN ('pending', 'approved_by_vendor', 'rejected_by_vendor', 'escalated', 'app  
  ),  
  vendor_response TEXT,  
  admin_notes TEXT,  
  processed_by UUID REFERENCES users(id), -- Admin who processed  
  requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  resolved_at TIMESTAMP  
);
```

```
CREATE INDEX idx_refunds_order ON refunds(order_id);  
CREATE INDEX idx_refunds_status ON refunds(status);
```

9. Vendor Payouts Table

```

CREATE TABLE vendor_payouts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vendor_id UUID REFERENCES vendors(id),
  payout_period VARCHAR(20) NOT NULL, -- e.g., '2026-01' for January 2026
  total_sales DECIMAL(15, 2) NOT NULL,
  platform_commission DECIMAL(15, 2) NOT NULL,
  subscription_fee DECIMAL(15, 2) NOT NULL,
  refund_deductions DECIMAL(15, 2) DEFAULT 0.00,
  net_payout DECIMAL(15, 2) NOT NULL,
  status VARCHAR(20) DEFAULT 'pending' CHECK (
    status IN ('pending', 'processing', 'completed', 'failed')
  ),
  payment_reference VARCHAR(255),
  processed_by UUID REFERENCES users(id),
  processed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_payouts_vendor ON vendor_payouts(vendor_id);
CREATE INDEX idx_payouts_period ON vendor_payouts(payout_period);

```

10. Addresses Table

```

CREATE TABLE addresses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  address_type VARCHAR(20) CHECK (address_type IN ('shipping', 'billing')),
  full_name VARCHAR(255),
  phone VARCHAR(20),
  address_line1 TEXT NOT NULL,
  address_line2 TEXT,
  city VARCHAR(100) NOT NULL,
  state VARCHAR(100) NOT NULL,
  postal_code VARCHAR(20),
  country VARCHAR(100) DEFAULT 'Nigeria',
  is_default BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_addresses_user ON addresses(user_id);

```

MongoDB Schemas

1. Products Collection


```

{
  "_id": "ObjectId",
  "vendorId": "UUID (references vendors table)",
  "name": "Samsung Galaxy S24 Ultra",
  "slug": "samsung-galaxy-s24-ultra",
  "description": "Latest flagship smartphone with AI features",
  "categoryId": "UUID (references categories table)",
  "categoryPath": ["Electronics", "Mobile Phones", "Smartphones"],
  "brand": "Samsung",
  "basePrice": 450000.00,
  "currency": "NGN",

  "variants": [
    {
      "skuId": "UUID",
      "attributes": {
        "color": "Titanium Black",
        "storage": "256GB",
        "ram": "12GB"
      },
      "price": 450000.00,
      "compareAtPrice": 500000.00,
      "inventory": {
        "quantity": 50,
        "lowStockThreshold": 5,
        "isInStock": true
      },
      "images": [
        "https://cdn.example.com/s24-black-1.jpg",
        "https://cdn.example.com/s24-black-2.jpg"
      ],
      "weight": 233,
      "dimensions": {
        "length": 162.3,
        "width": 79.0,
        "height": 8.6
      }
    },
    {
      "skuId": "UUID",
      "attributes": {
        "color": "Titanium Gray",
        "storage": "512GB",
        "ram": "12GB"
      },
      "price": 520000.00,

```

```

        "inventory": {
            "quantity": 30,
            "lowStockThreshold": 5,
            "isInStock": true
        },
        "images": ["..."]
    }
],

"specifications": {
    "Display": "6.8 inch Dynamic AMOLED 2X",
    "Processor": "Snapdragon 8 Gen 3",
    "Camera": "200MP + 50MP + 12MP + 10MP",
    "Battery": "5000mAh"
},

"shipping": {
    "isFreeShipping": false,
    "shippingFee": 2000.00,
    "estimatedDeliveryDays": 3,
    "courierService": "GIG Logistics"
},

"seo": {
    "metaTitle": "Samsung Galaxy S24 Ultra - Buy Online",
    "metaDescription": "Get the latest Samsung flagship...",
    "keywords": ["samsung", "s24 ultra", "flagship phone"]
},

"status": "active", // active, draft, out_of_stock, archived
"isVerified": true, // Admin verification for quality control
"totalSold": 150,
"totalViews": 5000,
"averageRating": 4.7,
"totalReviews": 45,

"createdAt": "ISODate",
"updatedAt": "ISODate"
}

```

Indexes:

```

db.products.createIndex({ vendorId: 1 });
db.products.createIndex({ slug: 1 }, { unique: true });
db.products.createIndex({ categoryId: 1 });
db.products.createIndex({ status: 1 });
db.products.createIndex({ name: "text", description: "text", brand: "text" }); // Full-text

```

```

db.products.createIndex({ averageRating: -1 });
db.products.createIndex({ totalSold: -1 });
db.products.createIndex({ createdAt: -1 });

```

2. Product Reviews Collection

```

{
  "_id": "ObjectId",
  "productId": "ObjectId (references products)",
  "skuId": "UUID",
  "orderId": "UUID (references orders table)",
  "orderItemId": "UUID (references order_items table)",
  "customerId": "UUID (references users table)",
  "vendorId": "UUID (references vendors table)",

  "ratings": {
    "product": 5,      // 1-5 stars
    "vendor": 4,      // 1-5 stars
    "delivery": 5     // 1-5 stars
  },

  "review": {
    "title": "Excellent phone!",
    "content": "Best purchase I've made this year. Camera quality is outstanding.",
    "pros": ["Great camera", "Fast processor", "Premium build"],
    "cons": ["Expensive", "Heavy"]
  },

  "images": [
    "https://cdn.example.com/reviews/user-photo-1.jpg"
  ],

  "isVerifiedPurchase": true,
  "helpfulCount": 25,
  "reportCount": 0,
  "status": "approved", // pending, approved, rejected, flagged

  "vendorResponse": {
    "message": "Thank you for your feedback!",
    "respondedAt": "ISODate"
  },

  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}

```

Indexes:

```
db.reviews.createIndex({ productId: 1 });
db.reviews.createIndex({ customerId: 1 });
db.reviews.createIndex({ vendorId: 1 });
db.reviews.createIndex({ orderId: 1 });
db.reviews.createIndex({ "ratings.product": -1 });
db.reviews.createIndex({ createdAt: -1 });
```

3. Shopping Carts Collection (MongoDB or Redis)

```
{
  "_id": "ObjectId",
  "sessionId": "UUID", // For guest users
  "userId": "UUID",    // For logged-in users (NULL for guests)

  "items": [
    {
      "productId": "ObjectId",
      "skuId": "UUID",
      "productName": "Samsung Galaxy S24 Ultra",
      "skuAttributes": {
        "color": "Titanium Black",
        "storage": "256GB"
      },
      "price": 450000.00,
      "quantity": 1,
      "vendorId": "UUID",
      "imageUrl": "https://cdn.example.com/s24-thumb.jpg",
      "addedAt": "ISODate"
    }
  ],

  "subtotal": 450000.00,
  "totalItems": 1,
  "expiresAt": "ISODate", // Auto-delete after 7 days of inactivity
  "updatedAt": "ISODate"
}
```

4. Notifications Collection

```
{
  "_id": "ObjectId",
  "userId": "UUID",
  "type": "order_shipped", // order_placed, order_shipped, order_delivered, refund_approved,
  "title": "Your order has been shipped!",
  "message": "Order #ORD-20260216-0001 is on its way.",
}
```

```

    "data": {
      "orderId": "UUID",
      "orderNumber": "ORD-20260216-0001",
      "actionUrl": "/orders/ORD-20260216-0001"
    },
    "channels": ["email", "in_app", "push"],
    "isRead": false,
    "sentAt": "ISODate",
    "createdAt": "ISODate"
  }

```

4. REST API Design

Base URL

Production: <https://api.marketplace.com/v1>

Development: <http://localhost:3000/api/v1>

Authentication

- JWT Bearer Token in Authorization header
 - Token format: Authorization: Bearer <token>
 - Token expiry: 24 hours (access token), 7 days (refresh token)
-

Module 1: Authentication & Users

POST /auth/register Description: Register new customer account

Access: Public

Request:

```

{
  "email": "akdavid@example.com",
  "password": "SecurePass123!",
  "firstName": "Ak",
  "lastName": "David",
  "phone": "+2348012345678"
}

```

Response: 201 Created

```

{
  "success": true,
  "message": "Account created. Please verify your email.",
  "data": {
    "user": {

```

```

        "id": "uuid",
        "email": "akdavid@example.com",
        "firstName": "Ak",
        "lastName": "David",
        "role": "customer",
        "isEmailVerified": false
    },
    "tokens": {
        "accessToken": "jwt_token",
        "refreshToken": "refresh_token",
        "expiresIn": 86400
    }
}

```

POST /auth/login **Description:** Login for all user types

Access: Public

Request:

```

{
  "email": "akdavid@example.com",
  "password": "SecurePass123!"
}

```

Response: 200 OK

```

{
  "success": true,
  "data": {
    "user": {
      "id": "uuid",
      "email": "akdavid@example.com",
      "role": "customer"
    },
    "tokens": {
      "accessToken": "jwt_token",
      "refreshToken": "refresh_token"
    }
  }
}

```

POST /auth/refresh-token **Description:** Get new access token using refresh token

Access: Public

Request:

```
{  
  "refreshToken": "refresh_token_here"  
}
```

POST /auth/logout **Description:** Logout and invalidate tokens

Access: Authenticated

POST /auth/verify-email **Description:** Verify email with token from email

Request:

```
{  
  "token": "email_verification_token"  
}
```

POST /auth/forgot-password **Description:** Request password reset email

Request:

```
{  
  "email": "akdavid@example.com"  
}
```

POST /auth/reset-password **Description:** Reset password with token

Request:

```
{  
  "token": "reset_token",  
  "newPassword": "NewSecurePass123!"  
}
```

GET /users/me **Description:** Get current user profile

Access: Authenticated

Response: 200 OK

```
{  
  "success": true,  
  "data": {
```

```
    "id": "uuid",
    "email": "akdavid@example.com",
    "firstName": "Ak",
    "lastName": "David",
    "phone": "+2348012345678",
    "role": "customer",
    "isEmailVerified": true,
    "createdAt": "2026-01-15T10:30:00Z"
  }
}
```

PATCH /users/me **Description:** Update user profile

Access: Authenticated

Request:

```
{
  "firstName": "Ak",
  "lastName": "David Jr",
  "phone": "+2348098765432"
}
```

POST /users/me/addresses **Description:** Add new address

Access: Authenticated

Request:

```
{
  "addressType": "shipping",
  "fullName": "Ak David",
  "phone": "+2348012345678",
  "addressLine1": "15 Admiralty Way",
  "addressLine2": "Lekki Phase 1",
  "city": "Lagos",
  "state": "Lagos",
  "postalCode": "101245",
  "isDefault": true
}
```

GET /users/me/addresses **Description:** Get all user addresses

Access: Authenticated

PATCH /users/me/addresses/:id **Description:** Update address
Access: Authenticated

DELETE /users/me/addresses/:id **Description:** Delete address
Access: Authenticated

Module 2: Vendor Management

POST /vendors/register **Description:** Register as vendor (creates vendor profile)

Access: Authenticated Customer

Request:

```
{
  "businessName": "Ak Tech Store",
  "businessEmail": "business@aktech.com",
  "businessPhone": "+2348012345678",
  "businessAddress": "15 Admiralty Way, Lekki, Lagos",
  "businessRegistrationNumber": "RC123456",
  "taxId": "TIN-123456789",
  "bankAccountName": "Ak Tech Store",
  "bankAccountNumber": "0123456789",
  "bankName": "GTBank"
}
```

Response: 201 Created

```
{
  "success": true,
  "message": "Vendor application submitted. Awaiting admin approval.",
  "data": {
    "vendorId": "uuid",
    "businessName": "Ak Tech Store",
    "verificationStatus": "pending"
  }
}
```

POST /vendors/:vendorId/documents **Description:** Upload vendor verification documents

Access: Vendor Owner

Request: multipart/form-data

documentType: business_license

file: (binary)

GET /vendors/me **Description:** Get vendor dashboard overview

Access: Vendor

Response: 200 OK

```
{
  "success": true,
  "data": {
    "vendor": {
      "id": "uuid",
      "businessName": "Ak Tech Store",
      "verificationStatus": "approved",
      "subscriptionPlan": "premium",
      "subscriptionEndDate": "2026-12-31",
      "commissionRate": 5.0,
      "rating": 4.7,
      "totalReviews": 230
    },
    "stats": {
      "totalProducts": 45,
      "totalSales": 2500000.00,
      "pendingOrders": 8,
      "thisMonthSales": 450000.00,
      "pendingPayout": 425000.00
    }
  }
}
```

PATCH /vendors/me **Description:** Update vendor profile

Access: Vendor

GET /vendors/me/analytics **Description:** Get detailed vendor analytics

Access: Vendor

Query Params: ?startDate=2026-01-01&endDate=2026-01-31

Response:

```
{
  "success": true,
  "data": {
    "salesByDay": [
      { "date": "2026-01-15", "sales": 45000.00, "orders": 5 }
    ]
  }
}
```

```

    ],
    "topProducts": [
      { "productId": "xxx", "name": "Samsung S24", "sold": 20, "revenue": 9000000.00 }
    ],
    "conversionRate": 3.5,
    "averageOrderValue": 180000.00
  }
}

```

GET /vendors/me/payouts **Description:** Get payout history

Access: Vendor

Response:

```

{
  "success": true,
  "data": {
    "payouts": [
      {
        "id": "uuid",
        "payoutPeriod": "2026-01",
        "totalSales": 2500000.00,
        "platformCommission": 125000.00,
        "subscriptionFee": 50000.00,
        "refundDeductions": 20000.00,
        "netPayout": 2305000.00,
        "status": "completed",
        "processedAt": "2026-02-01T10:00:00Z"
      }
    ],
    "nextPayout": {
      "period": "2026-02",
      "estimatedAmount": 450000.00,
      "payoutDate": "2026-03-01"
    }
  }
}

```

GET /vendors (Admin Only) **Description:** List all vendors with filters

Access: Admin

Query Params: ?status=pending&page=1&limit=20

PATCH /vendors/:vendorId/verify (Admin Only) Description: Approve/reject vendor application

Access: Admin

Request:

```
{
  "action": "approve", // or "reject"
  "rejectionReason": "Incomplete documents" // if rejecting
}
```

PATCH /vendors/:vendorId/subscription (Admin Only) Description: Update vendor subscription

Access: Admin

Request:

```
{
  "subscriptionPlan": "enterprise",
  "subscriptionEndDate": "2027-12-31",
  "commissionRate": 3.0
}
```

Module 3: Products

POST /products Description: Create new product

Access: Vendor

Request:

```
{
  "name": "Samsung Galaxy S24 Ultra",
  "description": "Latest flagship smartphone",
  "categoryId": "uuid",
  "brand": "Samsung",
  "basePrice": 450000.00,
  "variants": [
    {
      "attributes": {
        "color": "Titanium Black",
        "storage": "256GB"
      },
      "price": 450000.00,
      "inventory": {
        "quantity": 50,
        "lowStockThreshold": 5
      },
    },
  ],
}
```

```

        "images": ["url1", "url2"]
    }
],
"specifications": {
    "Display": "6.8 inch AMOLED",
    "Processor": "Snapdragon 8 Gen 3"
},
"shipping": {
    "isFreeShipping": false,
    "shippingFee": 2000.00,
    "estimatedDeliveryDays": 3
}
}

```

Response: 201 Created

```

{
  "success": true,
  "message": "Product created successfully. Awaiting admin verification.",
  "data": {
    "productId": "mongodb_id",
    "name": "Samsung Galaxy S24 Ultra",
    "slug": "samsung-galaxy-s24-ultra",
    "status": "draft"
  }
}

```

GET /products **Description:** List products with filters

Access: Public

Query Params:

```

?page=1
&limit=20
&categoryId=uuid
&minPrice=100000
&maxPrice=500000
&brand=Samsung
&sort=-createdAt (or -totalSold, -averageRating)
&search=galaxy

```

Response: 200 OK

```

{
  "success": true,
  "data": {
    "products": [

```

```

{
  {
    "id": "mongodb_id",
    "name": "Samsung Galaxy S24 Ultra",
    "slug": "samsung-galaxy-s24-ultra",
    "basePrice": 450000.00,
    "priceRange": {
      "min": 450000.00,
      "max": 520000.00
    },
    "mainImage": "url",
    "averageRating": 4.7,
    "totalReviews": 45,
    "totalSold": 150,
    "vendor": {
      "id": "uuid",
      "businessName": "Ak Tech Store",
      "rating": 4.8
    }
  }
},
{
  "pagination": {
    "currentPage": 1,
    "totalPages": 10,
    "totalItems": 200,
    "itemsPerPage": 20
  }
}
}

```

GET /products/:slug **Description:** Get single product details
Access: Public
Response: 200 OK (Full product object from MongoDB)

PATCH /products/:productId **Description:** Update product
Access: Vendor (owner only)

DELETE /products/:productId **Description:** Soft delete product
Access: Vendor (owner only)

PATCH /products/:productId/inventory **Description:** Update inventory for specific SKU

Access: Vendor

Request:

```
{
  "skuId": "uuid",
  "quantity": 100
}
```

POST /products/:productId/images **Description:** Upload product images

Access: Vendor

Request: multipart/form-data

GET /products/:productId/reviews **Description:** Get product reviews

Access: Public

Query Params: ?page=1&limit=10&sort=-createdAt&rating=5

Module 4: Categories

POST /categories (Admin Only) **Description:** Create main category

Access: Admin

Request:

```
{
  "name": "Electronics",
  "description": "Electronic devices and accessories",
  "imageUrl": "url"
}
```

POST /categories/:parentId/subcategories **Description:** Create subcategory

Access: Admin or Vendor (depending on hybrid rules)

Request:

```
{
  "name": "Mobile Phones",
  "description": "Smartphones and feature phones"
}
```

GET /categories **Description:** Get category tree

Access: Public

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "Electronics",
      "slug": "electronics",
      "children": [
        {
          "id": "uuid",
          "name": "Mobile Phones",
          "slug": "mobile-phones"
        }
      ]
    }
  ]
}
```

PATCH /categories/:categoryId **Description:** Update category

Access: Admin

DELETE /categories/:categoryId **Description:** Delete category (if no products)

Access: Admin

Module 5: Shopping Cart

POST /cart/items **Description:** Add item to cart

Access: Public (guest or authenticated)

Request:

```
{
  "productId": "mongodb_id",
  "skuId": "uuid",
  "quantity": 1
}
```

Response: 201 Created


```
{
  "success": true,
  "message": "Item added to cart",
  "data": {
    "cart": {
      "items": [
        {
          "productId": "mongodb_id",
          "skuId": "uuid",
          "productName": "Samsung Galaxy S24 Ultra",
          "skuAttributes": { "color": "Black", "storage": "256GB" },
          "price": 450000.00,
          "quantity": 1,
          "imageUrl": "url",
          "vendorId": "uuid"
        }
      ],
      "subtotal": 450000.00,
      "totalItems": 1
    }
  }
}
```

GET /cart **Description:** Get current cart
Access: Public

PATCH /cart/items/:itemId **Description:** Update cart item quantity
Access: Public
Request:

```
{
  "quantity": 2
}
```

DELETE /cart/items/:itemId **Description:** Remove item from cart
Access: Public

DELETE /cart **Description:** Clear entire cart
Access: Public

Module 6: Checkout & Orders

POST /checkout/validate **Description:** Validate cart before checkout
(check stock, prices)

Access: Public

Response:

```
{
  "success": true,
  "data": {
    "isValid": true,
    "warnings": [
      {
        "productId": "xxx",
        "message": "Only 3 items left in stock"
      }
    ],
    "orderSummary": {
      "subtotal": 450000.00,
      "shippingFee": 2000.00,
      "total": 452000.00
    }
  }
}
```

POST /orders **Description:** Create order

Access: Public (guest or authenticated)

Request:

```
{
  "customerEmail": "akdavid@example.com",
  "customerPhone": "+2348012345678",
  "shippingAddress": {
    "fullName": "Ak David",
    "phone": "+2348012345678",
    "addressLine1": "15 Admiralty Way",
    "city": "Lagos",
    "state": "Lagos",
    "postalCode": "101245"
  },
  "paymentMethod": "card",
  "createAccount": false // For guest checkouts
}
```

Response: 201 Created

```
{
  "success": true,
  "message": "Order created successfully",
  "data": {
    "order": {
      "id": "uuid",
      "orderNumber": "ORD-20260216-0001",
      "totalAmount": 452000.00,
      "paymentStatus": "pending",
      "orderStatus": "pending"
    },
    "paymentUrl": "https://paystack.com/pay/xyz" // If card payment
  }
}
```

GET /orders/:orderNumber Description: Get order details

Access: Customer (owner), Vendor (if vendor's items), Admin

Response: 200 OK

```
{
  "success": true,
  "data": {
    "order": {
      "id": "uuid",
      "orderNumber": "ORD-20260216-0001",
      "customerEmail": "akdavid@example.com",
      "totalAmount": 452000.00,
      "paymentMethod": "card",
      "paymentStatus": "paid",
      "orderStatus": "processing",
      "shippingAddress": {...},
      "createdAt": "2026-02-16T10:30:00Z"
    },
    "items": [
      {
        "id": "uuid",
        "productName": "Samsung Galaxy S24 Ultra",
        "skuAttributes": { "color": "Black", "storage": "256GB" },
        "quantity": 1,
        "unitPrice": 450000.00,
        "subtotal": 450000.00,
        "vendor": {
          "id": "uuid",

```

```
        "businessName": "Ak Tech Store"
      },
      "deliveryStatus": "processing",
      "trackingNumber": null
    }
  ]
}
```

GET /orders **Description:** List user's orders
Access: Authenticated
Query Params: ?status=delivered&page=1&limit=10

POST /orders/:orderId/cancel **Description:** Cancel order (before processing)
Access: Customer

GET /vendors/me/orders **Description:** Get vendor's orders
Access: Vendor
Query Params: ?deliveryStatus=pending&page=1

PATCH /orders/:orderId/items/:itemId/ship **Description:** Mark order item as shipped
Access: Vendor
Request:

```
{
  "trackingNumber": "TRACK123456",
  "courierService": "GIG Logistics"
}
```

PATCH /orders/:orderId/items/:itemId/deliver **Description:** Mark order item as delivered
Access: Vendor or System (courier webhook)
Request:

```
{
  "deliveredAt": "2026-02-20T15:30:00Z",
```

```
    "receivedBy": "Ak David"
  }
```

Module 7: Payments

POST /payments/initialize **Description:** Initialize payment (Paystack/Flutterwave)

Access: Customer

Request:

```
{
  "orderId": "uuid",
  "paymentMethod": "card",
  "paymentGateway": "paystack"
}
```

Response:

```
{
  "success": true,
  "data": {
    "paymentUrl": "https://checkout.paystack.com/xyz",
    "reference": "PSK_ref_123456"
  }
}
```

POST /payments/webhook/paystack **Description:** Paystack webhook for payment confirmation

Access: Paystack only (verify with signature)

POST /payments/webhook/flutterwave **Description:** Flutterwave webhook

Access: Flutterwave only

POST /payments/verify/:reference **Description:** Verify payment status

Access: Customer

Response:

```
{
  "success": true,
  "data": {
    "status": "successful",
  }
}
```

```
    "amount": 452000.00,  
    "paidAt": "2026-02-16T11:00:00Z"  
  }  
}
```

Module 8: Refunds & Disputes

POST /refunds **Description:** Request refund
Access: Customer
Request:

```
{  
  "orderId": "uuid",  
  "refundReason": "Product defective",  
  "refundType": "full"  
}
```

GET /refunds **Description:** Get user's refund requests
Access: Customer

GET /vendors/me/refunds **Description:** Get refund requests for vendor's products
Access: Vendor

PATCH /refunds/:refundId/respond **Description:** Vendor responds to refund
Access: Vendor
Request:

```
{  
  "action": "approve", // or "reject"  
  "vendorResponse": "We'll process the refund within 3 days"  
}
```

PATCH /refunds/:refundId/escalate **Description:** Escalate dispute to admin
Access: Customer

PATCH /refunds/:refundId/resolve (Admin Only) **Description:** Admin resolves escalated dispute

Access: Admin

Request:

```
{
  "decision": "approved", // or "rejected"
  "adminNotes": "Customer provided valid proof"
}
```

Module 9: Reviews & Ratings

POST /reviews **Description:** Submit review

Access: Customer (verified purchase only)

Request:

```
{
  "orderId": "uuid",
  "ratings": {
    "product": 5,
    "vendor": 4,
    "delivery": 5
  },
  "review": {
    "title": "Excellent phone!",
    "content": "Best purchase this year.",
    "pros": ["Great camera", "Fast"],
    "cons": ["Expensive"]
  }
}
```

GET /products/:productId/reviews **Description:** Get product reviews

Access: Public

GET /vendors/:vendorId/reviews **Description:** Get vendor reviews

Access: Public

POST /reviews/:reviewId/helpful **Description:** Mark review as helpful

Access: Authenticated

POST /reviews/:reviewId/report **Description:** Report inappropriate review

Access: Authenticated

POST /reviews/:reviewId/vendor-response **Description:** Vendor responds to review

Access: Vendor

Request:

```
{
  "message": "Thank you for your feedback! We're glad you love it."
}
```

Module 10: Search

GET /search **Description:** Global search

Access: Public

Query Params:

```
?q=samsung galaxy
&type=products (or vendors, or both)
&categoryId=uuid
&minPrice=100000
&maxPrice=500000
&page=1
&limit=20
```

GET /search/suggestions **Description:** Search autocomplete

Access: Public

Query Params: ?q=sams

Response:

```
{
  "success": true,
  "data": {
    "suggestions": [
      "Samsung Galaxy S24",
      "Samsung TV",
      "Samsung Watch"
    ]
  }
}
```

Module 11: Admin

GET /admin/dashboard **Description:** Admin dashboard statistics

Access: Admin

Response:

```
{
  "success": true,
  "data": {
    "overview": {
      "totalUsers": 50000,
      "totalVendors": 1200,
      "totalProducts": 45000,
      "totalOrders": 125000,
      "totalRevenue": 5000000000.00,
      "platformCommission": 250000000.00
    },
    "pendingActions": {
      "vendorApprovals": 15,
      "productVerifications": 50,
      "escalatedDisputes": 8
    }
  }
}
```

GET /admin/vendors **Description:** List all vendors

Access: Admin

PATCH /admin/vendors/:vendorId/suspend **Description:** Suspend vendor account

Access: Admin

GET /admin/products **Description:** List all products with moderation

Access: Admin

PATCH /admin/products/:productId/verify **Description:** Approve/reject product

Access: Admin

GET /admin/orders **Description:** View all orders
Access: Admin

GET /admin/refunds **Description:** View all refunds
Access: Admin

GET /admin/analytics **Description:** Platform analytics
Access: Admin
Query Params: ?startDate=2026-01-01&endDate=2026-01-31&metric=sales

POST /admin/payouts/process **Description:** Trigger monthly payout processing
Access: Admin

Module 12: Notifications

GET /notifications **Description:** Get user notifications
Access: Authenticated
Query Params: ?isRead=false&page=1&limit=20

PATCH /notifications/:notificationId/read **Description:** Mark notification as read
Access: Authenticated

PATCH /notifications/mark-all-read **Description:** Mark all notifications as read
Access: Authenticated

GET /notifications/preferences **Description:** Get notification preferences
Access: Authenticated

PATCH /notifications/preferences **Description:** Update notification preferences

Access: Authenticated

Request:

```
{
  "emailNotifications": {
    "orderUpdates": true,
    "promotions": false,
    "newReviews": true
  },
  "pushNotifications": {
    "orderUpdates": true,
    "promotions": false
  }
}
```

5. Module Structure (NestJS)

```
marketplace-backend/
  src/
    main.ts
    app.module.ts
    config/
      database.config.ts      # PostgreSQL + MongoDB config
      redis.config.ts
      jwt.config.ts
      payment.config.ts

    common/
      guards/
        jwt-auth.guard.ts
        roles.guard.ts
        vendor-owner.guard.ts
      decorators/
        roles.decorator.ts
        current-user.decorator.ts
        current-vendor.decorator.ts
      filters/
        http-exception.filter.ts
      interceptors/
        logging.interceptor.ts
        transform.interceptor.ts
      pipes/
        validation.pipe.ts
```

```

    utils/
        currency.util.ts      # Naira conversion
        pagination.util.ts
        slug.util.ts

modules/

    auth/
        auth.module.ts
        auth.controller.ts
        auth.service.ts
        strategies/
            jwt.strategy.ts
            refresh-token.strategy.ts
        dto/
            register.dto.ts
            login.dto.ts
            reset-password.dto.ts

    users/
        users.module.ts
        users.controller.ts
        users.service.ts
        entities/
            user.entity.ts      # TypeORM entity
            address.entity.ts
        dto/
            create-user.dto.ts
            update-user.dto.ts
            create-address.dto.ts

    vendors/
        vendors.module.ts
        vendors.controller.ts
        vendors.service.ts
        entities/
            vendor.entity.ts
            vendor-document.entity.ts
        dto/
            register-vendor.dto.ts
            update-vendor.dto.ts
            verify-vendor.dto.ts

    products/
        products.module.ts
        products.controller.ts

```

```

products.service.ts
schemas/
  product.schema.ts    # Mongoose schema
dto/
  create-product.dto.ts
  update-product.dto.ts
  filter-products.dto.ts

categories/
  categories.module.ts
  categories.controller.ts
  categories.service.ts
entities/
  category.entity.ts
dto/
  create-category.dto.ts

cart/
  cart.module.ts
  cart.controller.ts
  cart.service.ts
schemas/
  cart.schema.ts        # MongoDB or Redis
dto/
  add-to-cart.dto.ts

orders/
  orders.module.ts
  orders.controller.ts
  orders.service.ts
entities/
  order.entity.ts
  order-item.entity.ts
dto/
  create-order.dto.ts
  update-order-status.dto.ts

payments/
  payments.module.ts
  payments.controller.ts
  payments.service.ts
entities/
  payment.entity.ts
gateways/
  paystack.gateway.ts
  flutterwave.gateway.ts

```

```

    dto/
        initialize-payment.dto.ts

refunds/
    refunds.module.ts
    refunds.controller.ts
    refunds.service.ts
    entities/
        refund.entity.ts
    dto/
        create-refund.dto.ts
        respond-refund.dto.ts

reviews/
    reviews.module.ts
    reviews.controller.ts
    reviews.service.ts
    schemas/
        review.schema.ts    # MongoDB
    dto/
        create-review.dto.ts

payouts/
    payouts.module.ts
    payouts.controller.ts
    payouts.service.ts
    entities/
        vendor-payout.entity.ts
    jobs/
        monthly-payout.job.ts    # Cron job

notifications/
    notifications.module.ts
    notifications.controller.ts
    notifications.service.ts
    schemas/
        notification.schema.ts
    gateways/
        email.gateway.ts    # SendGrid
        sms.gateway.ts      # Termii
        push.gateway.ts     # Firebase
    templates/
        order-confirmation.hbs
        order-shipped.hbs

search/

```

```
    search.module.ts
    search.controller.ts
    search.service.ts
    indexers/
        product.indexer.ts    # Elasticsearch indexing
        vendor.indexer.ts

media/
    media.module.ts
    media.controller.ts
    media.service.ts
    providers/
        s3.provider.ts
        cloudinary.provider.ts

analytics/
    analytics.module.ts
    analytics.controller.ts
    analytics.service.ts

admin/
    admin.module.ts
    admin.controller.ts
    admin.service.ts

database/
    postgres/
        migrations/
        seeds/
    mongodb/
        seeds/

test/
    unit/
    e2e/

.env.example
.gitignore
docker-compose.yml
package.json
tsconfig.json
README.md
```

6. Third-Party Integrations

6.1 Payment Gateways - Paystack Integration

File: `src/modules/payments/gateways/paystack.gateway.ts`

This file implements all payment operations with Paystack, including initialization, verification, webhook validation, and refunds.

Key Methods: - `initializeTransaction()` - Creates a payment session and returns checkout URL - `verifyTransaction()` - Confirms payment after user completes checkout - `validateWebhook()` - Ensures webhook requests are actually from Paystack - `refund()` - Processes refunds for approved returns

6.2 Email Service (SendGrid)

File: `src/modules/notifications/gateways/email.gateway.ts`

Handles all transactional emails using SendGrid's dynamic templates.

Email Types: - Order confirmation - Order shipped notification - Delivery confirmation - Refund approval/rejection - Vendor approval/rejection

6.3 SMS Service (Termii)

File: `src/modules/notifications/gateways/sms.gateway.ts`

Sends SMS notifications for critical events like order shipped and delivery updates.

6.4 Cloud Storage (AWS S3)

File: `src/modules/media/providers/s3.provider.ts`

Handles file uploads for: - Product images - Vendor verification documents - Review images - User profile pictures

6.5 Search (Elasticsearch)

File: `src/modules/search/indexers/product.indexer.ts`

Provides fast, full-text search across products with: - Fuzzy matching for typos - Multi-field search (name, description, brand) - Category filtering - Price range filtering - Sorting by relevance, popularity, rating

7. Security & Authentication

7.1 JWT Strategy

All protected routes use JWT Bearer tokens for authentication.

7.2 Role-Based Access Control (RBAC)

Three user roles with different permissions: - **Customer:** Can browse, purchase, review - **Vendor:** Can manage products, view orders, respond to reviews - **Admin:** Full platform control

7.3 Vendor Ownership Guard

Ensures vendors can only modify their own products and view their own orders.

7.4 Rate Limiting

- Global: 100 requests per 15 minutes per IP
 - Route-specific: 5 requests per minute for review submission
-

8. Deployment Strategy

8.1 Docker Setup

Complete Docker Compose configuration with: - NestJS application - PostgreSQL database - MongoDB database - Redis cache - Nginx load balancer

8.2 Environment Variables

Comprehensive `.env.example` file with all required configuration: - Database connections - JWT secrets - Payment gateway keys - Email/SMS service credentials - Cloud storage credentials

8.3 Production Deployment (AWS)

Recommended AWS architecture: - ECS Fargate for application (auto-scaling) - RDS PostgreSQL (Multi-AZ) - DocumentDB for MongoDB - ElastiCache Redis - Application Load Balancer - CloudFront CDN - S3 for media storage

8.4 CI/CD Pipeline

GitHub Actions workflow for automated testing and deployment: 1. Run tests on every push 2. Build Docker image 3. Push to ECR 4. Deploy to ECS

9. Currency Conversion Utility

File: `src/common/utils/currency.util.ts`

As requested by Ak David, this utility provides Naira conversion for all financial calculations.

Features: - USD to NGN conversion - NGN to USD conversion - Formatted display with ₦ symbol - Combined display showing both currencies

Example Usage:

```
const totalSales = 500; // USD
console.log(CurrencyUtil.displayWithConversion(totalSales));
// Output: $500.00 → ₦800,000.00 (Rate: ₦1,600/$1)
```

10. Next Steps

Phase 1: Foundation (Weeks 1-2)

1. Set up NestJS project structure
2. Configure PostgreSQL + MongoDB connections
3. Implement authentication module (JWT)
4. Create User and Vendor entities

Phase 2: Core Features (Weeks 3-6)

1. Product management (CRUD + variants)
2. Category management
3. Shopping cart (Redis-backed)
4. Order creation flow
5. Payment integration (Paystack)

Phase 3: Advanced Features (Weeks 7-10)

1. Refund & dispute system
2. Review & rating system
3. Vendor payout automation (cron job)
4. Search with Elasticsearch
5. Email & SMS notifications

Phase 4: Admin & Polish (Weeks 11-12)

1. Admin dashboard APIs
2. Analytics endpoints
3. Testing (unit + e2e)
4. Documentation
5. Deployment to AWS

Summary

This architecture provides:

Modular Monolith - Clean separation, can split to microservices later
Hybrid Database - PostgreSQL for transactions, MongoDB for flexibility
60+ REST APIs - Complete coverage of all features
Production-Ready - Security, caching, queues, monitoring
Scalable - Auto-scaling, load balancing, CDN
Payment Integrations - Paystack & Flutterwave ready
Naira Support - Currency conversion utility as requested
Detailed Documentation - Every module explained with code examples

Ready to start building, Ak David!

For any specific module implementation or code examples, just ask!